**TITLE:** **Accuracy of naive summation: error growth, conditioning and better algorithms**

**SUBJECT:** Aim, method and results of the analyses in version 2 of the `Naive-Summation` repository

**AUTHOR:** Muhammed İkbal Yılmaz (myucanlar@gmail.com)

## 1. Aim and scope

Naive (recursive) summation computes $S = \sum_{i=1}^{n} x_i$ as $s_0 = 0$, $s_k = s_{k-1} + x_k$. In floating-point arithmetic every addition is rounded, $\mathrm{fl}(a + b) = (a + b)(1 + \delta)$ with $|\delta| \leq \varepsilon$, where $\varepsilon = 2^{-53} \approx 1.1 \times 10^{-16}$ is the unit roundoff of double precision. The computed sum $\hat{S}$ therefore has an error $E = \hat{S} - S$. Version 1 of this project measured $E$ for random data and concluded that naive summation is "numerically unstable and unreliable". This note asks three narrower questions, each answered by one analysis:

1. **How does the error grow with $n$,** compared with the theoretical bound, and does that depend on the data?
2. **What makes the relative error $|E|/|S|$ large?** Version 1 found that it "fluctuates".
3. **What do better algorithms gain, and at what cost?** And how accurate was the reference sum of version 1?

## 2. Method

- **Exact errors.** Every error is computed exactly: $\hat{S}$ and $-x_1, \ldots, -x_n$ are added with `math.fsum`, which tracks the sum without intermediate rounding, and the result is rounded once. Comparing with a rounded reference sum instead would add up to half a unit in the last place to every error, which is as large as the error of the best algorithms. The unit tests check this against exact rational arithmetic.
- **Algorithms.** Naive summation (computed with `np.cumsum`, which the tests show is bit-for-bit the same as a Python loop and gives all partial sums at once); pairwise summation, which adds neighbours level by level; and Kahan's (1965) compensated summation, which carries the low-order part lost in each addition into the next.
- **Bounds** (Higham, 2002, ch. 4), with $\gamma_k = k\varepsilon/(1 - k\varepsilon)$: naive $|E| \leq \gamma_{n-1} \sum |x_i|$; pairwise $|E| \leq \gamma_{\lceil \log_2 n \rceil} \sum |x_i|$; Kahan $|E| \leq (2\varepsilon + O(n\varepsilon^2)) \sum |x_i|$. Every error measured in the unit tests stays within these bounds.
- **Data.** Uniform random numbers as in version 1, $\mathrm{U}(-1, 1)$, and for

comparison $\mathrm{U}(0,1)$, where all terms have the same sign. Seeds are fixed.

## 3. Analysis 1: Error growth (Figure 1)

**Why.** The bound for naive summation grows linearly in $n$, and version 1 described the error as growing linearly. But a bound is a worst case: it assumes that every rounding error has the largest size and the same sign. The question is how the error grows in practice.

**Method.** For each distribution, 30 arrays of $10^6$ values; for 26 lengths $n$ between 10 and $10^6$ the error on the first $n$ values is computed exactly. The figure shows the root mean square (RMS) of $|E|/(\varepsilon \sum |x_i|)$; in these units the naive bound is the line $n-1$.

**How to read the figure.** Both axes are logarithmic, so a quantity that grows like $n^k$ appears as a line of slope $k$. The distance between a curve and the dashed line shows how pessimistic the bound is.

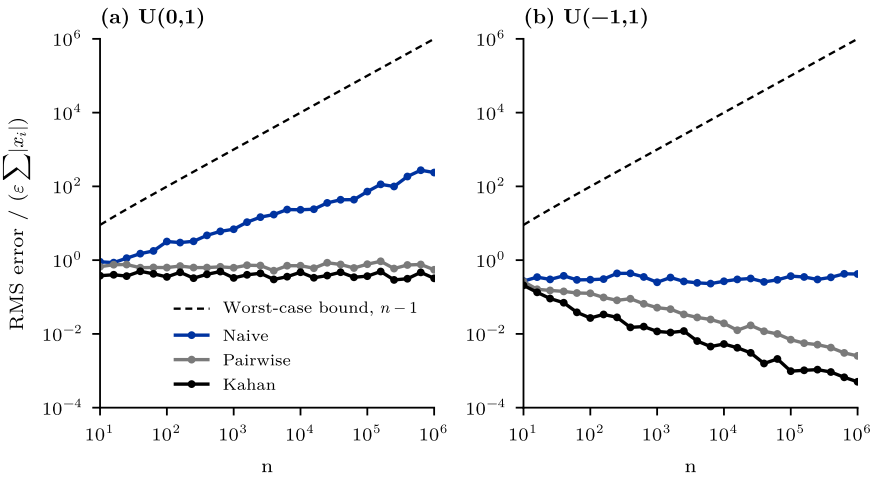



**Figure 1.** RMS error of three algorithms over 30 arrays, in units of $\varepsilon \sum |x_i|$. Dashed: worst-case bound for naive summation.

**Finding.** With positive terms, $\mathrm{U}(0,1)$, the naive error grows with slope 0.50 on this scale: like $\sqrt{n}$, as predicted when rounding errors behave like independent random signs (Johnson, 2019). At $n = 10^6$ it is 237 units, against $10^6$ for the bound. With terms of both signs, $\mathrm{U}(-1,1)$, the slope is only 0.06: the partial sums stay small, so each rounding error is small, and the error stays below one unit (0.42 at $n = 10^6$). In both cases pairwise and Kahan summation stay below one unit at every $n$ (at $n = 10^6$: 0.55 and 0.32 for $\mathrm{U}(0,1)$; 0.0025 and 0.0005 for $\mathrm{U}(-1,1)$).

**Interpretation.** "The error grows linearly" describes the bound, not the typical behaviour. How fast the error actually grows depends on the data, through the size of the partial sums.

## 4. Analysis 2: Conditioning (Figure 2)

**Why.** The relative error of version 1 varied strongly from run to run. Dividing the naive bound by $|S|$ gives $|E|/|S| \leq \gamma_{n-1}\,\kappa$ with $\kappa = \sum|x_i|/|\sum x_i|$, the condition number of the sum. When terms of both signs nearly cancel, $|S|$ is small and $\kappa$ is large, so even a small absolute error is a large relative error. This analysis checks whether $\kappa$ explains the variation.

**Method.** 2,000 arrays of $n = 10^4$ values from U$(-1, 1)$; for each, $\kappa$ and the exact relative error of naive and Kahan summation.

**How to read the figure.** Each point is one array. Points that rise to the right mean that the relative error follows the condition number.

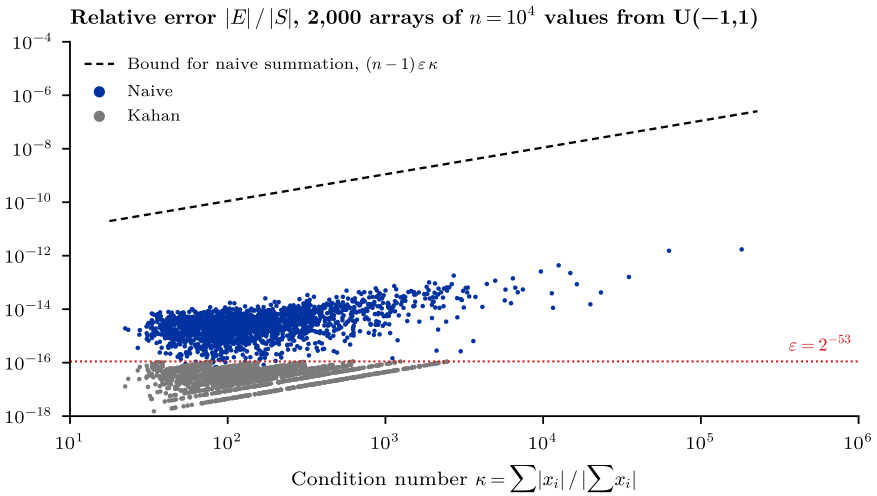



**Figure 2.** Relative error against the condition number for 2,000 arrays of $10^4$ values from U$(-1, 1)$. Dashed: bound for naive summation. Dotted: $\varepsilon$.

**Finding.** The median $\kappa$ is 125, the largest 181,407. The naive relative error rises with $\kappa$ (slope 0.64 on the log–log scale) and stays far below the bound (0 violations). Its mean, $7.6 \times 10^{-15}$, is 2.8 times its median, $2.7 \times 10^{-15}$, because the 1% of arrays with the largest errors account for 37% of the mean: an average of relative errors, as in version 1, is dominated by a few sums close to zero. Kahan's relative error never exceeds $1.00\,\varepsilon$, whatever $\kappa$; its points form bands because the result is within one rounding of the exact sum, and roundings come in discrete steps.

**Interpretation.** The "fluctuation" is not instability of the algorithm but the conditioning of the problem. Naive summation is backward stable: $\hat{S}$ is the exact sum of inputs perturbed by at most $\gamma_{n-1}$ in relative terms (Higham, 2002, ch. 4; Johnson, 2019). A large relative error in an ill-conditioned sum is what any backward stable method can

produce.

## 5. Analysis 3: Algorithms and the version 1 reference (Table 1)

**Why.** A more accurate algorithm is only worth using if its cost is acceptable, so accuracy and time are measured together. Version 1 used `np.sum(x, dtype=np.longdouble)` as the exact sum; the precision of `longdouble` varies across platforms (NumPy documentation), so its accuracy is measured as well.

**Method.** 10 arrays of $10^6$ values from $\mathrm{U}(-1, 1)$, as in version 1. Errors as in analysis 1; time is the median of five calls (Darwin arm64, Python 3.13.6, NumPy 2.4.1).

**Table 1.** RMS error in units of $\varepsilon \sum |x_i|$ and running time for $n = 10^6$.

| Method | RMS error | Time (ms) |
|---|---|---|
| Naive, Python loop (version 1) | 0.442 | 17.86 |
| Naive, `np.cumsum` | 0.442 | 2.09 |
| Pairwise | $1.9 \times 10^{-3}$ | 0.47 |
| `np.sum` (NumPy, partial pairwise) | $2.4 \times 10^{-3}$ | 0.13 |
| Kahan, Python loop | $4.2 \times 10^{-4}$ | 67.10 |
| `math.fsum` (correctly rounded) | $4.2 \times 10^{-4}$ | 15.13 |
| `np.sum`, `longdouble` (version 1 reference) | $2.4 \times 10^{-3}$ | 0.13 |

**Finding.** The Python loop of version 1 is both inaccurate and slow: it takes 17.9 ms, while `math.fsum`, whose result is the exact sum correctly rounded, takes 15.1 ms (1.2 times faster) and `np.sum` takes 0.13 ms with an error two orders of magnitude smaller. Kahan's method reaches the accuracy of `fsum` but is slow when written in Python (67 ms); its advantage is that it needs only a constant amount of extra work per term, which matters in compiled code. On this machine `longdouble` has the same precision as `float64`, so the version 1 reference is simply NumPy's pairwise sum in double precision; its error is 0.5% of the error of naive summation. Measuring errors against an exact reference, as done here, avoids this platform dependence.

## 6. Conclusions and limitations

- The error of naive summation is far below its worst-case bound. It grows like $\sqrt{n}$ on the bound's scale for terms of one sign and hardly grows for terms that cancel. The relative error is governed by the condition number.
- Naive summation is backward stable, not unstable. For accurate sums in Python, `math.fsum` is both correctly rounded and faster than a Python loop; `np.sum` is fast and much more accurate than naive summation.
- Only uniform data, double precision and sizes up to $10^6$ were studied.

Times were measured on a single machine.

## References

Higham, N. J. (1993). The accuracy of floating point summation. *SIAM Journal on Scientific Computing*, 14(4), 783–799.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.

Johnson, S. G. (2019). Notes on the accuracy of naive summation; and Backwards stability of recursive summation. MIT course 18.335J, *Introduction to Numerical Methods*, Lecture 3.

Kahan, W. (1965). Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1), 40.

NumPy documentation: `numpy.sum`, `numpy.cumsum`, `numpy.finfo`. Python documentation: `math.fsum`.

All numbers are generated from `figures/*.csv` by `make_paper.py`.